Protocol Document for Large Language Models for Architectural Analysis and Reasoning Support to Design Self-Adaptive Software Systems - A Systematic Literature Review

(Protocol Document)

Nadeem Abbas and Nazia Shahzadi

Linnaeus University, Sweden

{nadeem.abbas, nazia.shahzadi}@lnu.se

1. Introduction

Modern software systems operate under rapidly changing technologies, heterogeneous requirements, and interconnected environments, which increase the cognitive load on architects who must balance quality attributes and reason over large design spaces under uncertainty [1,2]. Architectural analysis, reasoning, and decision making are intrinsically complex: practitioners face a combinatorial design space, interdependent choices whose effects propagate across components and lifecycle phases, and incomplete or shifting information. This complexity is further compounded by multi-objective trade-offs (e.g., performance, reliability, security), organizational and regulatory constraints, and path dependence introduced by prior architectural decisions. Self-Adaptive Software Systems (SASS) provide feedback-control loops (often conceptualized as MAPE-K) and runtime configuration to maintain goals amidst workload, resource, and context variability [3,4]. Having runtime context-awareness and self-adaptation capabilities, SASS represents a class of *Context-Aware, Autonomous, and Smart Architecture (CASA)* systems. In software-intensive systems, architectural design involves analysis and reasoning.

Designing SASS heightens the need for rigorous architectural reasoning across runtime variability and design-time variability. Recent advances in Large Language Models (LLMs) open opportunities to augment architectural thinking. Evidence across Software Engineering (SE) tasks (e.g., requirements analysis, documentation, testing) suggests that LLMs can synthesize alternatives, articulate rationales, and navigate heterogeneous information. Early works on self-adaptation and architecture point to LLMs' potential for contextual interpretation, trade-off reasoning, and plan refinement. However, existing studies explore the use of LLMs either for developing self-adaptive systems or for supporting software architecture and design; they rarely examine their use in supporting architectural analysis and reasoning specifically for SASS or broader CASA systems and their product lines.

However, existing research on the use of LLMs in this context remains fragmented. Most studies to date focus either on LLM applications in software engineering more broadly or on self-adaptive systems without clearly addressing architectural reasoning and design. The intersection of LLM-supported architectural reasoning and the design of SASS is therefore underexplored, and there is limited consolidated knowledge regarding how LLMs are integrated architecturally, what roles they play in self-adaptation, and how their use is enhanced and evaluated [5,6].

To address the above-specified gap, this study reports a Systematic Literature Review (SLR) aimed at understanding the current landscape of LLM-based support for architectural analysis and reasoning in the design of SASS. The review is guided by the following RQs:

2. Research Questions

- **RQ1:** What has been reported to date on the use of LLMs for architectural analysis and reasoning support in the design of SASS and their product lines?
  Rationale: This question maps the state of the art, clarifying how and where LLMs are being applied in architectural tasks, what problems they address, and what limitations or gaps exist.
- **RQ2:** What techniques are reported to optimize and evaluate LLMs for architectural analysis and reasoning support in the design of SASS?
  Rationale: LLM-supported architectural reasoning is useful if the models produce accurate, grounded, and trustworthy results, particularly in adaptive systems. Examining existing optimization and evaluation techniques helps determine whether current methods are sufficient for these needs and highlights areas where improvement is critical. This makes RQ2 essential for advancing reliable LLM use in SASS design.
- **RQ3: What** are the emerging trends in the use of LLMs for architectural analysis and reasoning support in the design of SASS?
  Rationale: Understanding emerging trends helps anticipate future innovations, shaping the development of new tools, frameworks, and methodologies aligned with evolving practices to design and develop SASS.

These RQs define the scope and direction of the review and guide all subsequent activities, including search, selection, and data extraction.

3. Research Method

The study goal and RQs, specified above in Section 1, require a review of already published literature. Thus, we used the SLR method and followed the guidelines of Kitchenham et al. [7] to plan, conduct, and report the review. Following the guidelines, the SLR was done in three phases: planning, conducting, and reporting. The three phases are depicted in **Figure 1** and described below.

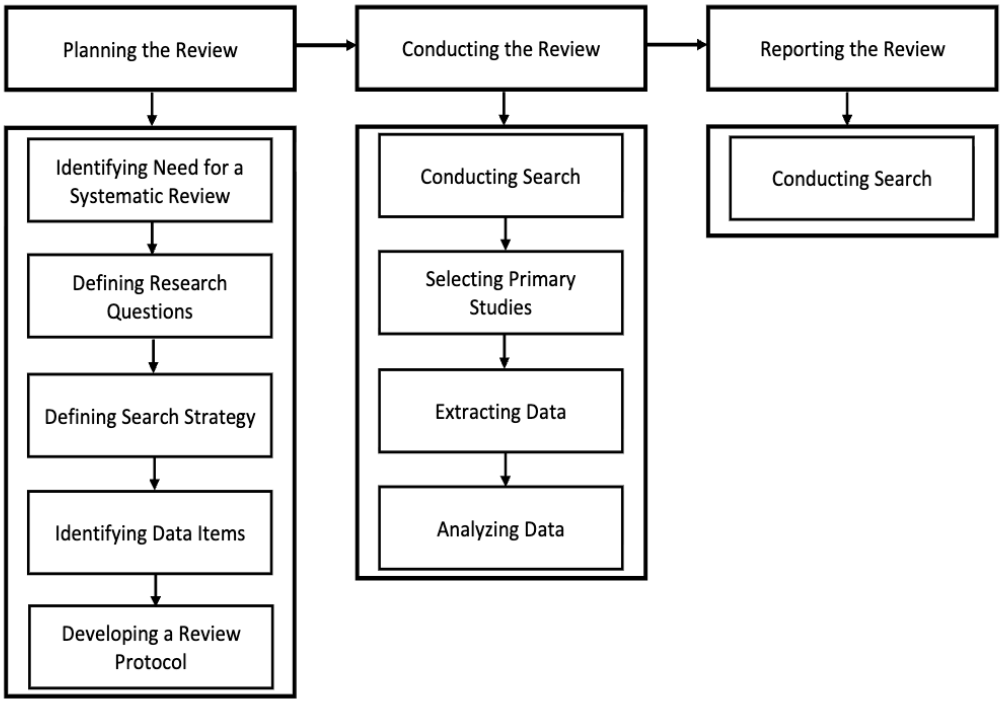

Figure 1: Systematic Literature Review Process based on guidelines given in [7,8].

4. Planning the Review

This section describes in detail how the review is planned and designed to ensure rigor, transparency, and reproducibility.

4.1 Identifying the Need for the Review

There is a research gap on the intersection of LLMs, software architecture design, and SASS. While there is research available on LLMs that is applied in SASS and SE tasks, their combined role is still unexplored and resulting in an immature domain. As a result, a SLR is essential for:

- Consolidation of existing research
- Assessment of current approaches
- Identification of limitations and challenges

4.2 Defining Research Questions

The guide for the entire process of review is based on the RQs that are defined in **Section 2.** They provide direction towards the data that needs to be collected, the evaluation of studies, and how the findings are synthesized.

4.3 Search Strategy

We have designed the search strategy to identify relevant studies in a comprehensive and systematic way so that biasness can be minimized.

4.3.1 Keyword Derivation and Variant Mapping

The search terms used in this review were systematically derived from the Research Questions (RQ1–RQ3) by identifying a core set of conceptual keywords, including *LLMs*, *software architecture*, *architectural analysis and reasoning*, *SASS*, *product lines*, *optimization techniques*, *evaluation methods*, and *emerging research directions*. Given the emerging and interdisciplinary nature of the research area, these cores keywords are expressed heterogeneously across the literature. Therefore, for each primary keyword, we identified a set of synonyms, variants, and closely related terms as they appear in the included studies. These variants include, for example, alternative formulations such as *reasoning plan generation*, *decision justification*, *dynamic reconfiguration*, *agentic systems*, *hybrid architectures*, *retrieval-augmented generation*, and *adaptive workflows*. All identified keyword variants were documented in a supporting file named ‘’Keywords_and_Similar_Words.xlxs’’ in the Replication_Package to ensure coverage of semantically equivalent terminology used across different research communities and application domains.

4.3.2 Search Approach

There are three approaches combined in the search process:

- **Automatic search** for retrieval of studies using the predefined search strings
- **Manual search** to keep the relevant studies and filter out the irrelevant ones
- **Snowballing** is implemented in (forward and backward) way to find out the additional studies [9,10]

The overall combination results in improved coverage of studies and reduces the risk of missing relevant studies.

4.3.3 Data Sources

The following libraries/databases are used to extract the data:

- IEEE Xplore
- Scopus
- ACM Digital Library

The reason for selecting these libraries is because the high-quality publications about SE, architecture design, and intelligent systems are covered by them.

4.3.4 Search String

We derived the search string by extracting the keywords from our RQs and combined them into following three categories:

- Terms related to LLMs
- Terms related to architectural design and reasoning
- Terms related to SASS

To form the final search string, the keywords are combined with Boolean operators. The details of search strings executed are as follows:

General Search String

("large language model" OR "large language models" OR LLM OR LLMs OR
"generative AI" OR "generative Artificial Intelligence" OR "Gen AI" OR
"Gen Artificial Intelligence" OR "generative model" OR
"Transformer model" OR "Transformer models" OR
GPT OR "GPT-4" OR "GPT-3" OR ChatGPT OR BERT OR RoBERTa OR T5
OR PaLM OR LLaMA OR Claude)
AND
("software architecture" OR "architecture design" OR "architectural design" OR
"architecture modeling" OR "architectural modeling" OR "architecture evaluation" OR
"architecture analysis" OR "architectural reasoning" OR "architecture decision-making" OR
"design rationale" OR "decision support" OR "architecture framework" OR
"architecture pattern" OR "architecture tactic" OR "trade-off analysis" OR
"quality attribute analysis" OR "design decision")
AND (adapt* OR self* OR autonom*)

Note: This is the general query that we generated by extracting keywords from our RQs.

IEEE Xplore Search String

(("Document Title":"large language model" OR "Document Title":"large language models" OR "Document Title":"LLMs" OR "Document Title":LLM OR "Document Title":"generative AI" OR "Document Title":"generative Artificial Intelligence" OR "Document Title":"Gen AI" OR "Document Title":"Gen Artificial Intelligence" OR "Document Title":"generative model" OR "Document Title":"Transformer models" OR "Document Title":"Transformer model" OR "Document Title":GPT OR "Document Title":"GPT-4" OR "Document Title":"GPT-3" OR "Document Title":"ChatGPT" OR "Document Title":"BERT" OR "Document Title":"RoBERTa" OR "Document Title":"T5" OR "Document Title":"PaLM" OR "Document Title":"LLaMA" OR "Document Title":"Claude" OR "Abstract":"large language model" OR "Abstract":"large language models" OR "Abstract":"LLMs" OR "Abstract":LLM OR "Abstract":"generative AI" OR "Abstract":"generative Artificial Intelligence" OR

"Abstract":"Gen AI" OR "Abstract":"Gen Artificial Intelligence" OR "Abstract":"generative model" OR "Abstract":"Transformer models" OR "Abstract":"Transformer model" OR "Abstract":GPT OR "Abstract":"GPT-4" OR "Abstract":"GPT-3" OR "Abstract":"ChatGPT" OR "Abstract":"BERT" OR "Abstract":"RoBERTa" OR "Abstract":"T5" OR "Abstract":"PaLM" OR "Abstract":"LLaMA" OR "Abstract":"Claude") AND ("Document Title":"software architecture" OR "Document Title":"architecture design" OR "Document Title":"architectural design" OR "Document Title":"architecture modeling" OR "Document Title":"architectural modeling" OR "Document Title":"architecture evaluation" OR "Document Title":"architecture analysis" OR "Document Title":"architectural reasoning" OR "Document Title":"architecture decision-making" OR "Document Title":"design rationale" OR "Document Title":"decision support" OR "Document Title":"architecture framework" OR "Document Title":"architecture pattern" OR "Document Title":"architecture tactic" OR "Document Title":"trade-off analysis" OR "Document Title":"quality attribute analysis" OR "Document Title":"design decision" OR "Abstract":"software architecture" OR "Abstract":"architecture design" OR "Abstract":"architectural design" OR "Abstract":"architecture modeling" OR "Abstract":"architectural modeling" OR "Abstract":"architecture evaluation" OR "Abstract":"architecture analysis" OR "Abstract":"architectural reasoning" OR "Abstract":"architecture decision-making" OR "Abstract":"design rationale" OR "Abstract":"decision support" OR "Abstract":"architecture framework" OR "Abstract":"architecture pattern" OR "Abstract":"architecture tactic" OR "Abstract":"trade-off analysis" OR "Abstract":"quality attribute analysis" OR "Abstract":"design decision") AND ("Document Title":adapt* OR "Document Title":self* OR "Document Title":autonom* OR "Abstract":adapt* OR "Abstract":self* OR "Abstract":autonom*))

Scopus Search String

(TITLE ("large language model" OR "large language models" OR "LLMs" OR LLM OR "generative AI" OR "generative Artificial Intelligence" OR "Gen AI" OR "Gen Artificial Intelligence" OR "generative model" OR "Transformer models" OR "Transformer model" OR GPT OR "GPT-4" OR "GPT-3" OR "ChatGPT" OR "BERT" OR "RoBERTa" OR "T5" OR "PaLM" OR "LLaMA" OR "Claude") OR ABS ("large language model" OR "large language models" OR "LLMs" OR LLM OR "generative AI" OR "generative Artificial Intelligence" OR "Gen AI" OR "Gen Artificial Intelligence" OR "generative model" OR "Transformer models" OR "Transformer model" OR GPT OR "GPT-4" OR "GPT-3" OR "ChatGPT" OR "BERT" OR "RoBERTa" OR "T5" OR "PaLM" OR "LLaMA" OR "Claude")) AND (TITLE ("software architecture" OR "architecture design" OR "architectural design" OR "architecture modeling" OR "architectural modeling" OR "architecture evaluation" OR "architecture analysis" OR "architectural reasoning" OR "architecture decision-making" OR "design rationale" OR "decision support" OR "architecture framework" OR "architecture pattern" OR "architecture tactic" OR "trade-off analysis" OR "quality attribute analysis" OR "design decision") OR ABS ("software architecture" OR "architecture design" OR "architectural design" OR "architecture modeling" OR "architectural modeling" OR "architecture evaluation" OR "architecture analysis" OR "architectural reasoning" OR "architecture decision-making" OR "design rationale" OR "decision support" OR "architecture framework" OR "architecture pattern" OR "architecture tactic" OR

"trade-off analysis" OR "quality attribute analysis" OR "design decision")) AND (TITLE (adapt* OR self* OR autonom*) OR ABS (adapt* OR self* OR autonom*))

ACM-DL Search String

(Title:"large language model" OR Title:"large language models" OR Title:"llms" OR Title:llm OR Title:"generative ai" OR Title:"generative artificial intelligence" OR Title:"gen ai" OR Title:"gen artificial intelligence" OR Title:"generative model" OR Title:"transformer models" OR Title:"transformer model" OR Title:gpt OR Title:"gpt-4" OR Title:"gpt-3" OR Title:"chatgpt" OR Title:"bert" OR Title:"roberta" OR Title:"t5" OR Title:"palm" OR Title:"llama" OR Title:"claude" OR Abstract:"large language model" OR Abstract:"large language models" OR Abstract:"llms" OR Abstract:llm OR Abstract:"generative ai" OR Abstract:"generative artificial intelligence" OR Abstract:"gen ai" OR Abstract:"gen artificial intelligence" OR Abstract:"generative model" OR Abstract:"transformer models" OR Abstract:"transformer model" OR Abstract:gpt OR Abstract:"gpt-4" OR Abstract:"gpt-3" OR Abstract:"chatgpt" OR Abstract:"bert" OR Abstract:"roberta" OR Abstract:"t5" OR Abstract:"palm" OR Abstract:"llama" OR Abstract:"claude")
AND
(Title:"software architecture" OR Title:"architecture design" OR Title:"architectural design" OR Title:"architecture modeling" OR Title:"architectural modeling" OR Title:"architecture evaluation" OR Title:"architecture analysis" OR Title:"architectural reasoning" OR Title:"architecture decision-making" OR Title:"design rationale" OR Title:"decision support" OR Title:"architecture framework" OR Title:"architecture pattern" OR Title:"architecture tactic" OR Title:"trade-off analysis" OR Title:"quality attribute analysis" OR Title:"design decision" OR Abstract:"software architecture" OR Abstract:"architecture design" OR Abstract:"architectural design" OR Abstract:"architecture modeling" OR Abstract:"architectural modeling" OR Abstract:"architecture evaluation" OR Abstract:"architecture analysis" OR Abstract:"architectural reasoning" OR Abstract:"architecture decision-making" OR Abstract:"design rationale" OR Abstract:"decision support" OR Abstract:"architecture framework" OR Abstract:"architecture pattern" OR Abstract:"architecture tactic" OR Abstract:"trade-off analysis" OR Abstract:"quality attribute analysis" OR Abstract:"design decision")
AND
(Title:adapt* OR Title:self* OR Title:autonom* OR Abstract:adapt* OR Abstract:self* OR Abstract:autonom*)

4.3.5 Search Validation

To validate the adequacy and coverage of the search string, we applied the Quasi Gold Standard (QGS) [10] method, a well-established approach for assessing search string effectiveness in systematic literature reviews. The QGS method evaluates whether a predefined set of known relevant studies (QGS articles) can be retrieved using the proposed search strategy, thereby providing confidence in its recall capability. When talking about the intersection of LLMs, software architecture, and SASS, these domains are quite immature, and emerging and existing studies did not cover all aspects of these domains together. To avoid these limitations, a pairwise concept validation approach is applied to the QGS, where the approach ensures coverage of all conceptual dimensions of LLMs, software architecture, and self-adaptation even when a study addresses only a subset of these dimensions explicitly.

A relevant pairwise combination of keyword group, included in the QGS of every search string, has been performed on every article instead of including every keyword at once. LLM-related and architecture-related components of the search string were applied for QGS articles primarily focusing on LLMs and software architecture design. Similarly, for QGS articles focusing on LLMs and self-adaptation, the LLM-related and self-adaptation-related components were used. This strategy reflects the fragmented nature of the current literature and allows validation of the search string's ability to retrieve studies spanning different conceptual intersections.

Table 1 presents the selected QGS articles along with the corresponding keyword groups used for their retrieval, indicating which conceptual dimensions of LLMs, software architecture, and self-adaptation are explicitly addressed by each article. The successful retrieval of all QGS articles through at least one relevant pairwise keyword combination provides confidence that the search string sufficiently captures the breadth of the research landscape while remaining sensitive to the emerging and heterogeneous nature of the field. The details of QGS articles are also given in Master_Index.xlxs with file name '' Quasi_Gold_Standard_Articles''.

Table 1: QGS Articles and the Keywords Parts used for Article Extraction

QGS Article	LLM Keywords	Architecture Keywords	Self-adaptation Keywords
Reimagining self-adaptation in the age of large language models [6]	✓	✗	✓
Can LLMs generate architectural design decisions? —An exploratory empirical study [14]	✓	✓	✗
Knowledge-Based Multi-Agent Framework for Automated Software Architecture Design [15]	✓	✓	✗
Self-Adaptive Large Language Model (LLM)-Based Multiagent Systems [16]	✓	✗	✓
Leveraging Self-Adaptive Systems and Generative AI for Personalizing Educational Serious Games: Architecture and Future Challenges [17]	✓	✗	✓

4.3.6 Time Span

Corresponding to the rapid development and emergence of LLMs, our search covers the studies published between January 2018 to October 2025, reflecting the emergence of transformer-based LLMs and subsequent applicability to software architecture and self-adaptation.

4.4 Study Selection Criteria

We have included/excluded the studies based on some criteria's given as follows:

4.4.1 Inclusion Criteria

Studies are included if they ensure that the selected articles contain the intersection of LLMs, SASS, and software architecture design so that the relevance of our research objective is maintained.

4.4.2 Exclusion Criteria

Studies are excluded if:

- There are categories other than conferences and journals
- They are not written in English, since it's a primary language for research community in SE
- If the articles are shorter than 5 pages in length

The criteria ensure relevance, quality, and sufficient detail.

4.5 Primary Study Selection Procedure

We have performed the primary study selection in multiple steps:

- Removal of duplicate studies
- Screening of title and abstract
- Full text review **(only if needed)**
- Conflicts were resolved among researchers with mutual discussion

The consistency and reduction of biasness is done with this multi-step process.

4.6 Data Extraction

The data extraction is performed following the steps given below:

4.6.1 Data Items

We extracted the data using predefined data items listed in **Table 2**.

Table 2: Data extraction items (D1–D18) and their purpose.

ID	Item Description	Purpose
D1-D4	Authors, year, title, venue	Documentation
D5	Quality score (Q1–Q6)	Quality
D6	LLM model used	RQ1
D7	Architectural activity supported	RQ1
D8	SASS lifecycle phase supported	RQ1
D9	Application domain	RQ1
D10	Role of LLM in architecture	RQ1
D11	Optimization technique	RQ2
D12	Evaluation method	RQ2
D13	Dataset / case study	RQ2
D14	Performance metrics	RQ2
D15	Reported limitations	RQ2
D16	Emerging trends	RQ3
D17	Research gaps	RQ3
D18	Future work	RQ3

The file containing data items is given in Master_Index.xlxs with name ‘’Data_Extraction_Items.xlxs’’.

4.6.2 Data Extraction Process

Data is extracted systematically and then recorded in a structured format (spreadsheets) to ensure traceability and consistency. The link to the study data is given here: Replication_Package.

4.7 Quality Assessment

Predefined criteria were set to evaluate the quality of primary studies. The score of criteria was from 0 to 2 and the total score was set to 12 for (Q1 to Q6). The details of quality assessment criteria are given in **Table 3**.

Table 3. Quality assessment items (Q1–Q6). Each item scored 0 (absent), 1 (general), or 2 (explicit); total 0–12 per study.

Item	Description
Q1	Problem definition (clarity and specificity).
Q2	Problem context (system/domain/assumptions).
Q3	Research design (how the study was conducted).
Q4	Contributions (claimed outputs/novelty).
Q5	Insights (findings/interpretation).

Q6	Limitations (threats/constraints).

Quality assessment helped in evaluation of studies in terms of their reliability and supported the weighted interpretation of results. The file Reasons_for_Assigned_Quality_Scores.pdf provides a transparent and traceable justification for all quality assessment decisions. The document provides the details for each of these three studies including the rationale behind the assigned scores for all six criteria's of quality (Q1–Q6) i.e. problem definition, problem context, research design, contributions, insights and limitations respectively. Each criterion is supported by an explicit reference to specific sections, pages, figures, and paragraphs from the original papers. These references provide support to the assigned scores, thereby ensuring auditability, replicability of process, and consistency. If there is an implicit discussion on the limitations, this is mentioned clearly and reflected with an appropriate score for that aspect. By connecting each score of quality to the solid evidence in the source studies, this artifact adds to the validity of the process and allows external verification and replication by other researchers.

5. Conducting the Review

The process of review follows the following steps:

5.1 Conducting Search

The predefined search string is executed among selected libraries in their specific formats (given in Section 4.3.3). They provide the required information regarding the primary research papers.

5.2 Selecting Primary Studies

The inclusion/exclusion criteria are applied (given in Section 4.4), to identify the primary studies.

5.3 Data Analysis

When we executed the search strings in all three libraries, we got the following results: We selected the following three libraries: ACM Digital Library, IEEE Xplore, and Scopus. The combination of three libraries was strategically used to minimize the risk of missing relevant studies, thereby strengthening the completeness and reliability of the evidence base. As the search engines used by these digital libraries differ in their query syntax, the search string was specialized and adapted for each library. Without using any filters, the search string produced **455** Scopus results, **20** ACM Digital Library results, and **75** IEEE Xplore results. We obtained **61** results from IEEE Xplore, **19** from ACM Digital Library, and **196** from Scopus after applying the filters (including conferences, journals, years between **2018 to October 2025**, and topics unique to computer science and engineering). So, the final number of articles was **276**. We next applied the inclusion/exclusion criteria to these articles and removed duplicates. **58** duplicates were found and **218** were the unique articles resulting in a final number of **03** primary articles.

Below is the list of **03** identified primary articles:

Title	DOI
(S1): An Architecture for Integrating Large Language Models with Digital Twins and Automation Systems [11]	10.1109/ETFA65518.2025.11205636
(S2): Improving Adaptability in Optimization-Based Decision Support Systems Through Large Language Models [12]	10.1109/ACCESS.2025.3601518
(**S3):** An Adaptive Multi-Agent LLM-Based Clinical Decision Support System Integrating Biomedical RAG and Web Intelligence [13]	10.1109/ACCESS.2025.3613340

The PDF files of three primary studies are also given in Replication_Package.

5.4 Overview of Identified Studies and Scope Extension

During the review process, the initial purpose was to focus exclusively on studies addressing the use of LLMs for architectural analysis and reasoning in the design of SASS. However, regardless of a wide and systematic search across major digital libraries, only three primary studies met all inclusion and quality criteria. This limited number of studies highlight that research at the intersection of LLMs, software architecture, and SASS is still at an early and fragmented stage. To better understand these findings and avoid drawing conclusions from an overly narrow evidence base, the scope of the study was expanded to include research on reflecting the emergence of transformer-based LLMs and subsequent applicability to software architecture and self-adaptation., even when the study is not focused on self-adaptation. The increase in scope enabled the identification of 64 additional architecture-focused studies that investigate the use of LLMs in architectural decision making, trade-off analysis, and design assistance. Although these studies do not explicitly address SASS, they do provide valuable understandings related to architectural insights, techniques, and patterns that can be helpful in integrating LLM-based reasoning into SASS/CASA systems. Accordingly, the expanded evidence base serves as a foundation for identifying trends, gaps, and opportunities for advancing architecture-centric LLM support in self-adaptive software systems.

5.5 Replication Package and Study Artifacts

The accompanying file named Master_Index constitutes the full set of study artifacts produced during the execution of the Systematic Literature Review. It consolidates all intermediate and results across multiple structured sub-sheets. Specifically, for example; it includes: (i) the complete data extraction schema (D1–D18) and extracted data for the three primary studies, covering bibliographic metadata, LLM models used, architectural activities supported, lifecycle phases, roles of LLMs, optimization and evaluation techniques, datasets, metrics, limitations, emerging trends, gaps, and future research directions; (ii) the complete results of the automated searches conducted in ACM Digital Library, IEEE Xplore, and Scopus, with explicit inclusion and exclusion decisions based on predefined criteria; (iii) detailed logs of duplicate identification and removal across all libraries, documenting the transition from raw search hits to unique candidate studies; (iv) the resulting set of unique articles after deduplication, forming the basis for screening;

(v) the definition and validation of the QGS articles, including the specific keyword subsets used to retrieve each QGS paper; (vi) comprehensive records of forward and backward snowballing, including cited and citing articles, accessibility checks, inclusion decisions, and reasons for exclusion; (vii) expanded file containing 64 results for the broader "LLMs and Software Architecture" scope used for contextualization with forward and backward snowballing results; and (viii) the final set of **t**hree primary studies selected for in-depth analysis, with explicit justifications for inclusion and architectural relevance to SASS. Together, these sub-sheets provide ***full transparency and traceability of the search,*** selection, validation, and analysis processes, enabling replication and extension of the review.

6. Reporting the Review Results

This section analyzes and synthesizes the findings derived from the three primary studies included in this SLR. We combine descriptive statistics with structured qualitative content analysis to answer RQ1–RQ3. Although the evidence base is small, all included studies exhibit high methodological quality; two studies scored 12 and one scored 11, supporting confidence in the extracted insights.

6.1. RQ1: Use of LLMs for Architectural Analysis and Reasoning Support in SASS

Overall, the reviewed studies suggest that research at the intersection of LLMs, software architecture, and SASS is in an exploratory phase. Rather than being treated as first-class architectural elements, LLMs are used as augmentative reasoning components that complement existing adaptive mechanisms. Across S1–S3, LLMs contribute to contextual interpretation, adaptive planning, anomaly detection, hypothesis validation, and explanation of generation activities foundational to SASS design-time and runtime reasoning. Study S1 proposes a three-layer architecture in which an LLM operates within a cognition layer tightly integrated with digital twins and physical automation systems. The LLM interprets structured state representations from the twin, detects anomalies, validates operational plans, and recommends adjustments activities that map to MAPE-K monitoring, analysis, and planning in cyber-physical adaptive control loops.

Study S2 targets design-time architectural decision support in optimization-based decision-support systems. The LLM dynamically proposes and adapts evaluation metrics, refines optimization logic, elicits and interprets high-level preferences, and generates natural-language rationales to aid multi-criteria trade-off reasoning during architectural design. Study S3 implements an LLM-centered, multi-agent clinical decision-support pipeline that combines biomedical Retrieval-Augmented Generation (RAG), web evidence verification, and confidence calibration. The architecture iteratively validates competing diagnostic hypotheses and refines decisions as an example of LLM-enabled runtime analysis and planning integrated into an adaptive feedback loop.

- Interpreting structured or unstructured contextual data (S1--S3)
- Generating candidate decisions, plans, or configurations (S1--S3)

- Validating outputs through multi-source grounding (S3)
- Coordinating sub-tasks within a multi-agent architecture (S3)
- Providing human-readable rationale for architectural decisions (S2, S3)

However, key architectural concerns remain unaddressed. None of the studies examine LLM support for variability management, dynamic software product lines, or systematic architectural trade-off analysis despite these being central to SASS and their product lines. Similarly, no work addresses how LLMs can be embedded in unified architecture-centric frameworks that span the full MAPE-K loop. These gaps highlight promising directions for future research.

6.2. RQ2: Optimization and Evaluation Techniques for LLM-Based Architectural Reasoning

All three studies use distinct techniques to optimize LLM reasoning, mitigate hallucinations, and evaluate outputs. The S1 study highlights the importance of structured prompting and grounding based on digital twins for improved situational reasoning. The multi-agent decomposition apporach decreases the cognitive load on one LLM while the iterative reasoning approach helps in combating the hallucination by gradually improving the output. While its evaluation is primarily qualitative, case studies in robotic control and simulation provide initial evidence of feasibility.

The methods such as few short prompting, dynamic metric creation, self-debugging, and validation by LLM-as-a-judge are used by Study S2. These assessments include the evaluation of model correctness and robustness on dynamic vehicle routing problems. Results show both strengths and weaknesses of the study. The strengths include adaptability under changing problem constraints and sensitivity to prompt structure, and scalability limitations are the weaknesses of the study. Biomedical RAG, web evidence retrieval, specialization of multi-agents, and loop-based confidence aware techniques are utilized by Study S3. Benchmarking against domain standards and ablation studies are used for evaluation, and they offer an example of comparatively rigorous LLM evaluation in safety-critical settings.

Several cross-study themes emerge:

- Grounding strategies including retrieval-augmented generation, digital-twin state representations, and domain-specific metric generation are essential for reducing hallucinations and ensuring trustworthiness (S1--S3).
- Multi-agent reasoning is favored when tasks require specialized knowledge or iterative decomposition (S3).
- Iterative refinement: methods to support dynamic correction and self-improvement of reasoning outputs (S1, S3).
- Evaluation practices remain inconsistent, domain-specific, and largely ad hoc, making cross-study comparison difficult (S1--S3).

The lack of standardized evaluation approaches for LLM-assisted architectural reasoning represents an important research challenge. Future efforts should develop domain-agnostic benchmarks and metrics that assess architectural correctness, robustness to runtime changes,

explanation quality, and grounding reliability. Such benchmarks would improve reproducibility and support systematic comparisons across emerging LLM-based architectural approaches.

6.3. RQ3: Emerging Trends and Research Gaps

Despite their different domains and objectives, the studies reveal converging trends in the use of LLMs for SASS reasoning:

- Hybrid architectures: All studies combine LLMs with established systems such as digital twins (S1), RAG pipelines (S3), or optimization engines (S2).
- Multi-agent reasoning: Multi-agent orchestration is used to distribute reasoning tasks and improve interpretability (S3).
- Grounded reasoning: Retrieval-based grounding (S3), structured digital twin models (S1), and domain-specific metrics (S2) reduce hallucinations and enhance trustworthiness.
- Adaptive refinement loops: Iterative optimization, dynamic re-querying, and self-debugging are increasingly used to enhance output dependability.
- Explainability and human oversight: All studies provide human-readable justifications and require human validation for final decisions.

Several significant research gaps were exposed during the analysis:

- A lack of SASS-specific architectural frameworks that integrate LLMs across the full MAPE-K loop.
- Minimal attention to formal verification, safety assurance, and trustworthiness of LLM-driven architectural reasoning.
- Absence of research on LLM support for variability management and product-line reasoning.
- No standardized benchmarks to assess architectural reasoning performance, robustness, or interpretability.
- Limited validation in real-world or safety-critical domains where adaptive systems are most essential.

These gaps can be addressed by coordinated efforts across the software architecture, and self-adaptive systems of communities. Future research should seek to systematize early promising methods such as digital-twin grounding, multi-agent orchestration, and meta-reasoning into reusable architectural patterns. Merging the reasoning capabilities of LLMs with formal methods and runtime assurance mechanisms is critical for deploying such systems in trustworthy, safety-sensitive contexts.

7. Synthesis of Findings

Table 4 details how the three primary studies relate to RQ1--RQ3. In general, the findings present a picture of a promising but immature research landscape. LLMs currently support isolated analysis and reasoning tasks at design time and runtime, but they are far from being integrated into comprehensive architecture-centric frameworks for SASS. Hybrid, grounded, and multi-agent

approaches demonstrate strong potential, yet substantial challenges remain including reliability, explainability, and rigorous evaluation. Advancing this field will require developing architecture-centric integration frameworks, establishing standardized evaluation methodologies, exploring LLM-enabled variability reasoning, and combining LLMs with formal assurance techniques.

Table 4: Mapping of the three primary studies to RQ1–RQ3

RQs	S1	S2	S3
RQ1	LLM cognition layer over digital twins; system state interpretation, anomaly detection, plan validation (runtime MAPE-K).	LLM supports design-time trade-off reasoning by generating/adapting metrics, refining optimization logic, and explaining decisions.	LLM-driven multi-agent reasoning for clinical CDSS; contextual interpretation, hypothesis validation, and iterative runtime planning.
RQ2	Structured prompting + digital-twin grounding; (agentic) validation; qualitative evaluation via robotic/simulation cases.	Few-shot prompting, dynamic metric generation, self-debugging, LLM-as-a-judge; evaluated on VRP with robustness tests.	Biomedical RAG, web verification, role specialization, confidence-aware loops; medical QA benchmarks + ablation.
RQ3	LLM–digital twin integration for runtime situational awareness; agentic automation mirroring adaptive control loops.	LLM-driven adaptability and meta-reasoning for MCDM; potential ties to DSPL/variability in design-time reasoning.	Grounded reasoning and multi-agent orchestration; need for trustworthy and explainable LLM reasoning in safety-critical settings.

The primary study overall provides valuable early insights into how LLMs can enhance architectural reasoning and runtime adaptation, but substantial methodological, architectural, and validation-focused advancements are necessary before LLM-driven architectural support for SASS becomes robust and mature. Future research should prioritize:

- Constructing architecture-centric LLM integration frameworks for SASS,
- Developing standardized evaluation methods and benchmark datasets,
- Exploring LLM-driven support for variability management and product line engineering,
- Combining LLMs with formal methods for trustworthy reasoning.

As an important direction for advancing this emerging research area, we also recognize that the very limited number of studies explicitly targeting LLM-supported architectural analysis and reasoning for SASS leaves substantial untapped evidence in adjacent domains. Many existing studies work on the use of LLMs software architecture tasks more broadly or for developing self-adaptive systems. But they do not explicitly connect these capabilities to architectural reasoning in SASS and their product lines. This lack of direct focus explains why only three studies met our inclusion criteria despite the growing body of research on LLM-assisted software engineering and adaptive systems.

A natural next step is taken to address these gaps. The systematic review studies investigate LLM use either for software architecture or for the engineering of self-adaptive systems. The goal is to analyze how the concepts, reasoning mechanisms, and integration techniques reported in these broader areas can be adapted or extended to support architectural analysis and reasoning specific to SASS. As a first step in this regard, we identified several studies on architectural design using

the LLM and planning to increase the studies by deriving a set of principles that can help future architectural reasoning guided by LLMs in adaptive systems.

7. Threats to Validity

The main threats to the validity of this SLR, and the measures taken to mitigate them are discussed in this section. Following established guidelines for secondary studies [19,20], threats are organized around search, selection, data extraction, synthesis, and external validity. These threats are particularly relevant given the novelty of the research area and the small number of primary studies identified.

Search Validity ensures how well the chosen search string has retrieved relevant research articles. Terminology in the LLM and SASS domains evolves rapidly, and relevant work may use alternative expressions (e.g., "AI agents", "reasoning copilot"). To address this issue, we identified the keywords that directly come from the RQs, validated by the search string using the technique of QGS. This improved the language with trial searches and complemented the automatic search process with manual screening and backward–forward snowballing.

Selection Validity defines the chances of incorrectly included or excluded studies. As it is required to understand and interpret whether the selected papers cover the integration of LLMs, software architecture and SASS. Personal judgement sometimes leads to biases. This kind of threat is mitigated by following the predefined protocol which helps in making the decision along with conducting multi-stage screening and resolving uncertainties collaboratively among the authors.

Data Extraction Validity refers to the precision and consistency of the extracted data. There are chances of misinterpretation when identifying architectural roles, optimization approaches, or analysis of details. This kind of risk can be avoided by having at least two authors independently reviewing the extracted items, resolving discrepancies by consensus, and tracing all extracted information back to the original studies.

Synthesis Validity deals with the chances of biasness during interpretation and synthesizing of evidence. The limited number of studies may cause bias in recognizing the roles of architecture and procedures of optimization and future directions. This risk is addressed by limiting the synthesis to a specific criterion outlined in **Table 1 and Table 3,** which we jointly reviewed to ensure transparency, traceability, and consistency.

External Validity relates to the ability of generalization. These three studies cover different areas (clinical decision making, industrial automation, and optimization-based DSS), which may limit generalizability to other SASS contexts. Another possible reason for the small number of articles might be the publication bias because negative results are not published. To address this, we interpret the results as preliminary indicators of emerging trends, gaps, and research opportunities rather than definitive conclusions about mature practices in LLM-supported architectural reasoning for SASS.

References

1. Mazeiar Salehie and Ladan Tahvildari. Self-adaptive software: Landscape and research challenges. ACM transactions on autonomous and adaptive systems (TAAS), 4(2):1–42, 2009.
2. Nadeem Abbas and Jesper Andersson. Architectural reasoning support for product-lines of self-adaptive software systems-a case study. In European Con- ference on Software Architecture, pages 20–36. Springer, 2015.
3. Nadeem Abbas and Jesper Andersson. Architectural reasoning for dynamic software product lines. In Proceedings of the 17th International Software Product Line Conference Co-located Workshops, pages 117–124, 2013.
4. Nadeem Abbas and Jesper Andersson. Harnessing variability in product-lines of self-adaptive software systems. In Proceedings of the 19th International Conference on Software Product Line, pages 191–200, 2015.
5. Jialong Li, Mingyue Zhang, Nianyu Li, Danny Weyns, Zhi Jin, and Kenji Tei. Exploring the potential of large language models in self-adaptive systems. In Pro- ceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, pages 77–83, 2024.
6. Raghav Donakanti, Prakhar Jain, Shubham Kulkarni, and Karthik Vaidhyanathan. Reimagining self-adaptation in the age of large language models. In 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C), pages 171–174. IEEE, 2024.
7. Barbara Kitchenham, O Pearl Brereton, David Budgen, Mark Turner, John Bailey, and Stephen Linkman. Systematic literature reviews in software engineering–a systematic literature review. Information and software technology, 51(1):7–15, 2009.
8. Pearl Brereton, Barbara A Kitchenham, David Budgen, Mark Turner, and Mohamed Khalil. Lessons from applying the systematic literature review process within the software engineering domain. Journal of systems and software, 80(4):571–583, 2007.
9. Claes Wohlin. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In Proceedings of the 18th International Con- ference on Evaluation and Assessment in Software Engineering, EASE ’14, New York, NY, USA, 2014. Association for Computing Machinery.
10. He Zhang and Muhammad Ali Babar. On searching relevant studies in software engineering. 2010.
11. Yuchen Xia, Nasser Jazdi, and Michael Weyrich. An architecture for integrating large language models with digital twins and automation systems. In 2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA), pages 1–8. IEEE, 2025.
12. Gianpaolo Ghiani, Emanuele Manni, and Sandro Zacchino. Improving adaptability in optimization-based decision support systems through large language models. IEEE Access, 2025.
13. Çağatay Umut Öğdü, Kübra Arslanoğlu, and Mehmet Karaköse. An adaptive multi-agent llm-based clinical decision support system integrating biomedical rag and web intelligence. IEEE Access, 2025.
14. Dhar, Rudra, Karthik Vaidhyanathan, and Vasudeva Varma. "Can llms generate architectural design decisions?-an exploratory empirical study." In 2024 IEEE 21st International Conference on Software Architecture (ICSA), pp. 79-89. IEEE, 2024.

15. Zhang, Yiran, Ruiyin Li, Peng Liang, Weisong Sun, and Yang Liu. "Knowledge-based multi-agent framework for automated software architecture design." In Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, pp. 530-534. 2025.
16. Nascimento, Nathalia, Paulo Alencar, and Donald Cowan. "Self-adaptive large language model (llm)-based multiagent systems." In 2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C), pp. 104-109. IEEE, 2023.
17. Bucchiarone, Antonio, Federico Bonetti, and Enes Yigitbas. "Leveraging Self-Adaptive Systems and Generative AI for Personalizing Educational Serious Games: Architecture and Future Challenges." In 2025 IEEE/ACM 20th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), pp. 103-110. IEEE, 2025.